```
// === ДОБАВЛЯЕМ ОБЯЗАТЕЛЬНУЮ ВЕРШИННУЮ ТОЧКУ ВХОДА ДЛЯ DIRECTX 11 ===
struct VS_OUTPUT
{
   float4 Pos : SV_POSITION;
   float2 Tex : TEXCOORD0;
};

VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
{
   VS_OUTPUT output = (VS_OUTPUT)0;
   output.Pos = Pos;
   output.Tex = Tex;
   return output;
}

// ========================================================================
// ИСТИННЫЙ ОДНОПРОХОДНЫЙ ДЕФЕКТОСКОП АНАТОЛИЯ ФЕДОРОВИЧА (ОБРАЗЕЦ 2011 ГОДА)
// ========================================================================

// Входные ресурсы DirectX 11
Texture2D srcTexture : register(t0);       // Наш единственный сырой файл
Texture2D macroTexture : register(t1);     // Честно сжатая макроструктура 8x8 от GPU
SamplerState samplerState0 : register(s0);  // Point-сэмплер без замыливания

// Буфер констант (бывший register(c0))
cbuffer cbData : register(b0)
{
   float width;    // p0[0]
   float height;   // p0[1]
   float d_width;  // p0[2]
   float d_height; // p0[3]
   float splitX;   // Положение шторки
```

```
    float padding;
};

static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

// Вспомогательная функция чтения и перевода пикселя в монохром (tex2D)
float GetGray(float2 uv)
{
    return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
}

// Новая функция: читает пиксель напрямую по его матричному индексу
float GetMacroGray(int2 pos)
{
    // Зажимаем индексы от 0 до максимального размера сжатого буфера (width/8 - 1)
    // чтобы видеокарта никогда не выходила за границы памяти
    int max_x = (int)(width / 8.0f) - 1;
    int max_y = (int)(height / 8.0f) - 1;

    int cl_x = clamp(pos.x, 0, max_x);
    int cl_y = clamp(pos.y, 0, max_y);

    // .Load берет точную ячейку памяти. Третий параметр (0) - это мип-уровень.
    return macroTexture.Load(int3(cl_x, cl_y, 0)).r;
}

// 1. ЛЕВЫЙ ВЕРХНИЙ ПИКСЕЛЬ
float4 func_LT(float2 tex_in)
{
    float2 tex0_local = float2(tex_in.x - 2.0f * d_width, tex_in.y - 2.0f * d_height);
    int tex_x_base = (int)round(width * tex0_local.x / 4.0f);
    int tex_y_base = (int)round(height * tex0_local.y / 4.0f);
```

```
    int tex_x = tex_x_base / 2;
    int tex_y = tex_y_base / 2;

    int i_x = (tex_x_base % 2) * 4;
    int i_y = (tex_y_base % 2) * 4;

    float k1    = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S   = GetMacroGray(int2(tex_x,    tex_y));
    float k_LR  = GetMacroGray(int2(tex_x + 1, tex_y));
    float k_TB  = GetMacroGray(int2(tex_x,    tex_y + 1));
    float k_LRTB = GetMacroGray(int2(tex_x + 1, tex_y + 1));

    float k_SS = ((k_S * (9 - i_x) + k_LR * (7 - i_x)) * (9 - i_y) +
                (k_TB * (9 - i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

    float out_gray = saturate(k1 - k_SS + 0.5f);
    return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 2. ПРАВЫЙ ВЕРХНИЙ ПИКСЕЛЬ
float4 func_RT(float2 tex_in)
{
    float2 tex0_local = float2(tex_in.x - 2.0f * d_width, tex_in.y - 2.0f * d_height);
    int tex_x_base = (int)round(width * tex0_local.x / 4.0f);
    int tex_y_base = (int)round(height * tex0_local.y / 4.0f);

    int tex_x = tex_x_base / 2;
    int tex_y = tex_y_base / 2;

    int i_x = (tex_x_base % 2) * 4;
    int i_y = (tex_y_base % 2) * 4;
```

```
    float k1     = GetGray(tex_in + 2.0f * float2(-d_width, d_height));
    float k_S    = GetMacroGray(int2(tex_x,     tex_y));
    float k_LR   = GetMacroGray(int2(tex_x + 1, tex_y));
    float k_TB   = GetMacroGray(int2(tex_x,     tex_y + 1));
    float k_LRTB = GetMacroGray(int2(tex_x + 1, tex_y + 1));

    float k_SS = ((k_S * (9 - i_x) + k_LR * (7 - i_x)) * (9 - i_y) +
                  (k_TB * (9 - i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

    float out_gray = saturate(k1 - k_SS + 0.5f);
    return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 3. ЛЕВЫЙ НИЖНИЙ ПИКСЕЛЬ
float4 func_LB(float2 tex_in)
{
    float2 tex0_local = float2(tex_in.x - 2.0f * d_width, tex_in.y - 2.0f * d_height);
    int tex_x_base = (int)round(width * tex0_local.x / 4.0f);
    int tex_y_base = (int)round(height * tex0_local.y / 4.0f);

    int tex_x = tex_x_base / 2;
    int tex_y = tex_y_base / 2;

    int i_x = (tex_x_base % 2) * 4;
    int i_y = (tex_y_base % 2) * 4;

    float k1     = GetGray(tex_in + 2.0f * float2(d_width, -d_height));
    float k_S    = GetMacroGray(int2(tex_x,     tex_y));
    float k_LR   = GetMacroGray(int2(tex_x + 1, tex_y));
    float k_TB   = GetMacroGray(int2(tex_x,     tex_y + 1));
    float k_LRTB = GetMacroGray(int2(tex_x + 1, tex_y + 1));
```

```
    float k_SS = ((k_S * (9 - i_x) + k_LR * (7 - i_x)) * (9 - i_y) +
                  (k_TB * (9 - i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

    float out_gray = saturate(k1 - k_SS + 0.5f);
    return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 4. ПРАВЫЙ НИЖНИЙ ПИКСЕЛЬ
float4 func_RB(float2 tex_in)
{
    float2 tex0_local = float2(tex_in.x - 2.0f * d_width, tex_in.y - 2.0f * d_height);
    int tex_x_base = (int)round(width * tex0_local.x / 4.0f);
    int tex_y_base = (int)round(height * tex0_local.y / 4.0f);

    int tex_x = tex_x_base / 2;
    int tex_y = tex_y_base / 2;

    int i_x = (tex_x_base % 2) * 4;
    int i_y = (tex_y_base % 2) * 4;

    float k1    = GetGray(tex_in + 2.0f * float2(-d_width, -d_height));
    float k_S   = GetMacroGray(int2(tex_x,     tex_y));
    float k_LR  = GetMacroGray(int2(tex_x + 1, tex_y));
    float k_TB  = GetMacroGray(int2(tex_x,     tex_y + 1));
    float k_LRTB = GetMacroGray(int2(tex_x + 1, tex_y + 1));

    float k_SS = ((k_S * (9 - i_x) + k_LR * (7 - i_x)) * (9 - i_y) +
                  (k_TB * (9 - i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

    float out_gray = saturate(k1 - k_SS + 0.5f);
    return float4(out_gray, out_gray, out_gray, 1.0f);
```

```
}

// ГЛАВНЫЙ АППАРАТНЫЙ ШЕЙДЕР ФИЛЬТРАЦИИ
float4 PS_Stage3(VS_OUTPUT input) : SV_Target
{
   float2 tex0 = float2(input.Tex.x - 2.0f * d_width, input.Tex.y - 2.0f * d_height);
   int tex_x = round(width * tex0[0] / 4.0f);
   int tex_y = round(height * tex0[1] / 4.0f);

   if (tex_x % 2 == 0)
   {
      if (tex_y % 2 == 0) return func_LT(tex0);
      else                return func_LB(tex0);
   }
   else
   {
      if (tex_y % 2 == 0) return func_RT(tex0);
      else                return func_RB(tex0);
   }
}

// ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ ЭКРАНА С УЧЕТОМ ШТОРКИ ДО/ПОСЛЕ
Texture2D statsTexture : register(t1); // Сюда подадим готовый фильтр из Stage3

float4 PS_Final(VS_OUTPUT input) : SV_Target
{
   if (input.Tex.x < splitX)
   {
      // Слева переводим сырой оригинал в честный монохром по вашим весам
      float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
      float src_gray = dot(src_color.rgb, WEIGHT);
      return float4(src_gray, src_gray, src_gray, 1.0f);
```

```
    }

    // Тонкая разделительная черная линия
    if (abs(input.Tex.x - splitX) < 0.0015f) return float4(0.0f, 0.0f, 0.0f, 1.0f);

    // Справа выводим чистый результат дефектоскопа
    return statsTexture.Sample(samplerState0, input.Tex);
}

// Простой шейдер копирования кадра для ЭТАПА II
float4 PS_Copy(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// ВРЕМЕННАЯ ЗАГЛУШКА ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТА В MAIN.CPP
float4 PS(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// === ОФИЦИАЛЬНЫЕ ЗАГЛУШКИ ДЛЯ ЧИСТОГО СТАРТА ПРОГРАММЫ БЕЗ ОШИБОК ===

float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

float4 PS_Stage2(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}
```

```
float4 PS_PixelShaderStage2(VS_OUTPUT input) : SV_Target
{
   return srcTexture.Sample(samplerState0, input.Tex);
}

Texture2D    ss0 : register(t0);
SamplerState ssSampler : register(s0);

float4 main(VS_OUTPUT input) : SV_Target
{
   float2 tex0 = input.Tex;
   float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

   float GreyValue;
   float GrayMid = 0.0f;
   float GrayMin = 1.0f;
   float GrayMax = 0.0f;
   float GraySqr = 0.0f;

   for(int i = 0; i < 4; i++)
   {
      for(int j = 0; j < 4; j++)
      {
         float4 k1 = ss0.Sample(ssSampler, tex0);
         GreyValue = dot(k1.rgb, WEIGHT);

         GrayMid += GreyValue;
         GraySqr += GreyValue * GreyValue;

         if(GreyValue > GrayMax) GrayMax = GreyValue;
         if(GreyValue < GrayMin) GrayMin = GreyValue;
```

```
            tex0.x += d_width;
        }
        tex0.x -= d_width * 4.0f;
        tex0.y += d_height;
    }

    GrayMid /= 16.0f;
    GraySqr /= 16.0f;
    GraySqr -= GrayMid * GrayMid;
    if(GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

    return float4(GrayMid, GrayMin, GrayMax, GraySqr);
}

float4 PS_Downsample4x4(VS_OUTPUT input) : SV_Target
{
    float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

    float GreyValue;
    float GrayMid = 0.0f;
    float GrayMin = 1.0f;
    float GrayMax = 0.0f;
    float GraySqr = 0.0f;

    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < 4; j++)
        {
            float2 offset = float2(j * d_width, i * d_height);
            float4 k1 = srcTexture.Sample(samplerState0, input.Tex + offset);
```

```
            GreyValue = dot(k1.rgb, WEIGHT);

            GrayMid += GreyValue;
            GraySqr += GreyValue * GreyValue;

            if(GreyValue > GrayMax) GrayMax = GreyValue;
            if(GreyValue < GrayMin) GrayMin = GreyValue;
        }
    }

    GrayMid /= 16.0f;
    GraySqr /= 16.0f;
    GraySqr -= GrayMid * GrayMid;
    if(GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

    return float4(GrayMid, GrayMin, GrayMax, GraySqr);
}

float4 PS_Downsample8x8(VS_OUTPUT input) : SV_Target
{
    float4 k1;
    float GrayMid = 0.0f;
    float GrayMin = 1.0f;
    float GrayMax = 0.0f;
    float GraySqr = 0.0f;

    // Обходим блок 2x2 от результатов первой стадии (что дает суммарное сжатие 8x8)
    for(int i = 0; i < 2; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            // Читаем данные из текстуры первого шага (используем правильный сэмплер)
```

```
            float2 offset = float2(j * d_width * 4.0f, i * d_height * 4.0f);
            k1 = srcTexture.Sample(samplerState0, input.Tex + offset);

            GrayMid += k1.r; // Канал R хранит среднюю яркость блоков 4x4
            GraySqr += k1.w * k1.w + k1.r * k1.r; // Канал W хранит дисперсию

            if(k1.g < GrayMin) GrayMin = k1.g; // Канал G хранит минимум
            if(k1.b > GrayMax) GrayMax = k1.b; // Канал B хранит максимум
        }
    }

    GrayMid /= 4.0f;
    GraySqr /= 4.0f;
    GraySqr -= GrayMid * GrayMid;
    if(GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

    // Возвращаем финально упакованную макроструктуру 8x8
    return float4(GrayMid, GrayMin, GrayMax, GraySqr);
}
```